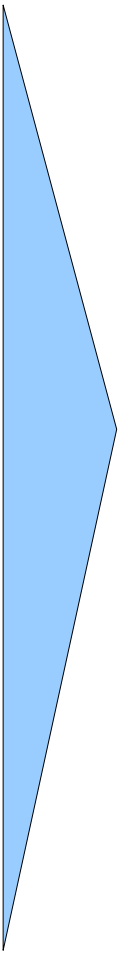
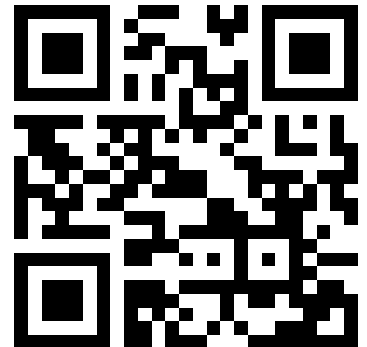


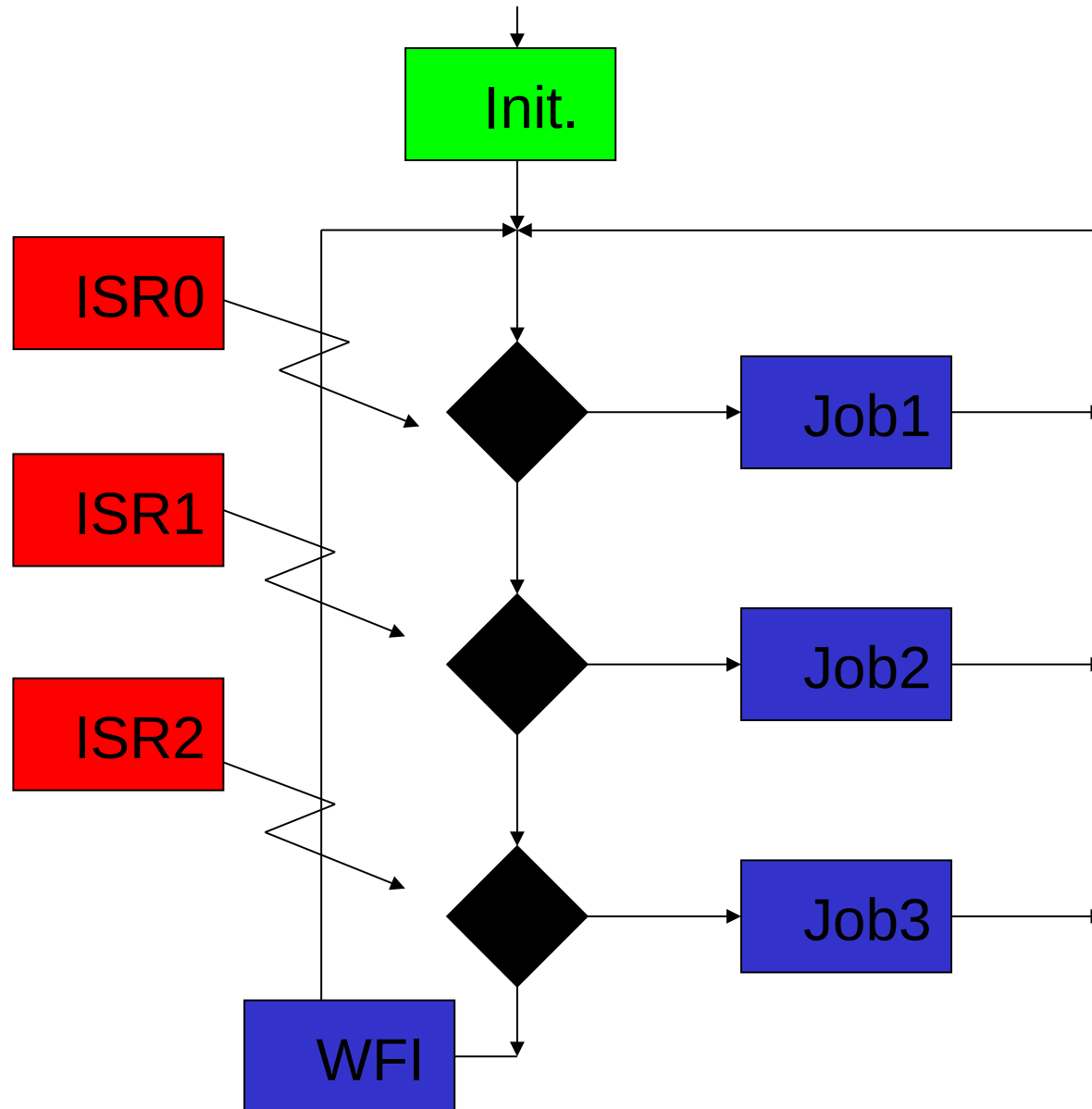
Lecture Overview

- 
- Introduction, Microcontroller basics
 - 32-bit Microcontroller technology
 - ARM Cortex M Architecture, (CM0 CM3 CM4 CM7)
 - Cortex M Microcontroller Instruction Set
 - Exception Handling, Interrupts
 - Real-time Operating Systems (RTOS)
 - Memory Management, protected memory, MPU
 - Implementing Calculations, Data Formats, FPU
 - I/O, Device drivers
 - Application Design
 - Case Studies, Advanced Design Patterns

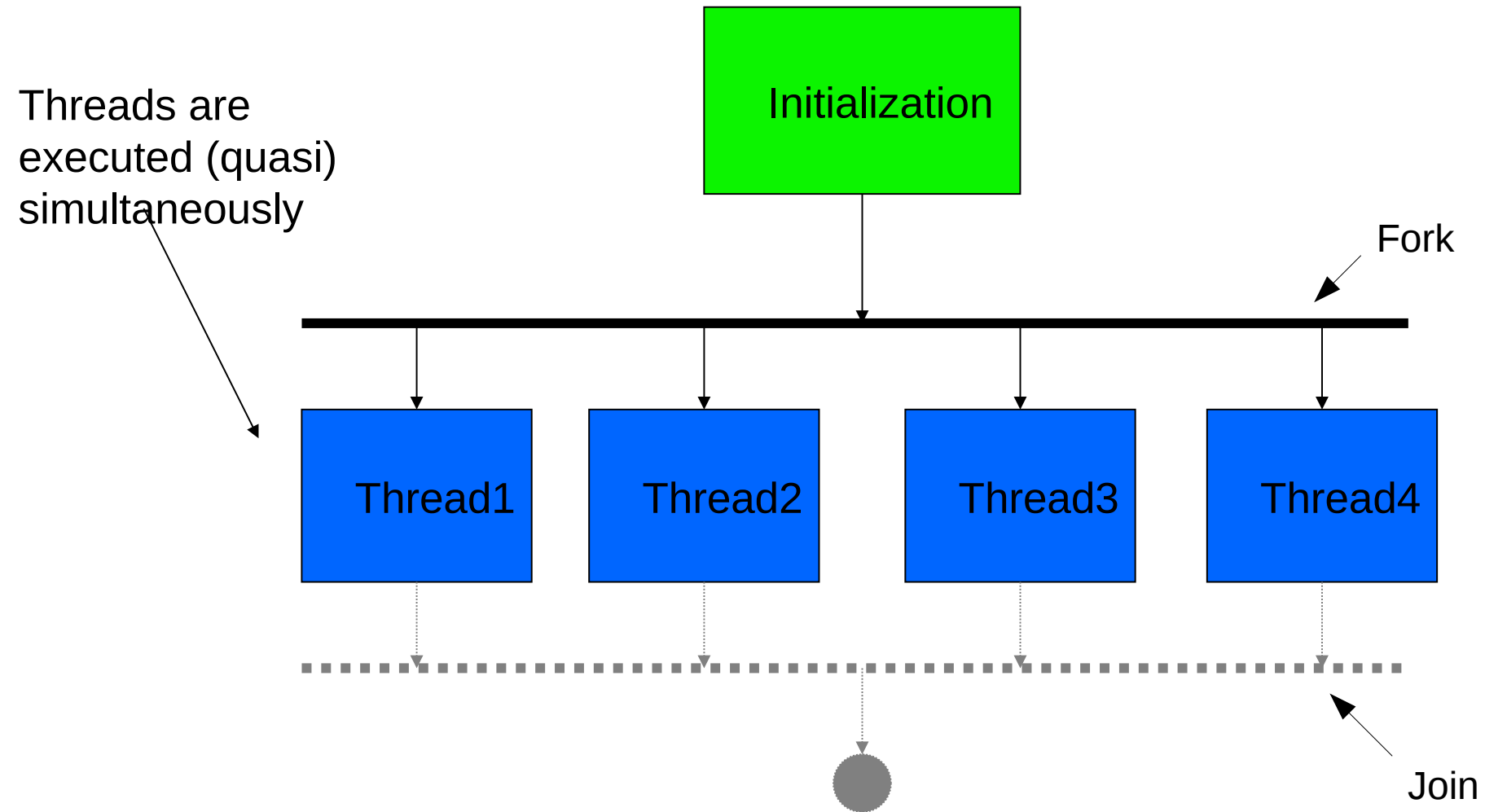
<https://skript.eit.h-da.de/ams/>



Interrupt-triggered Super Loop Architecture

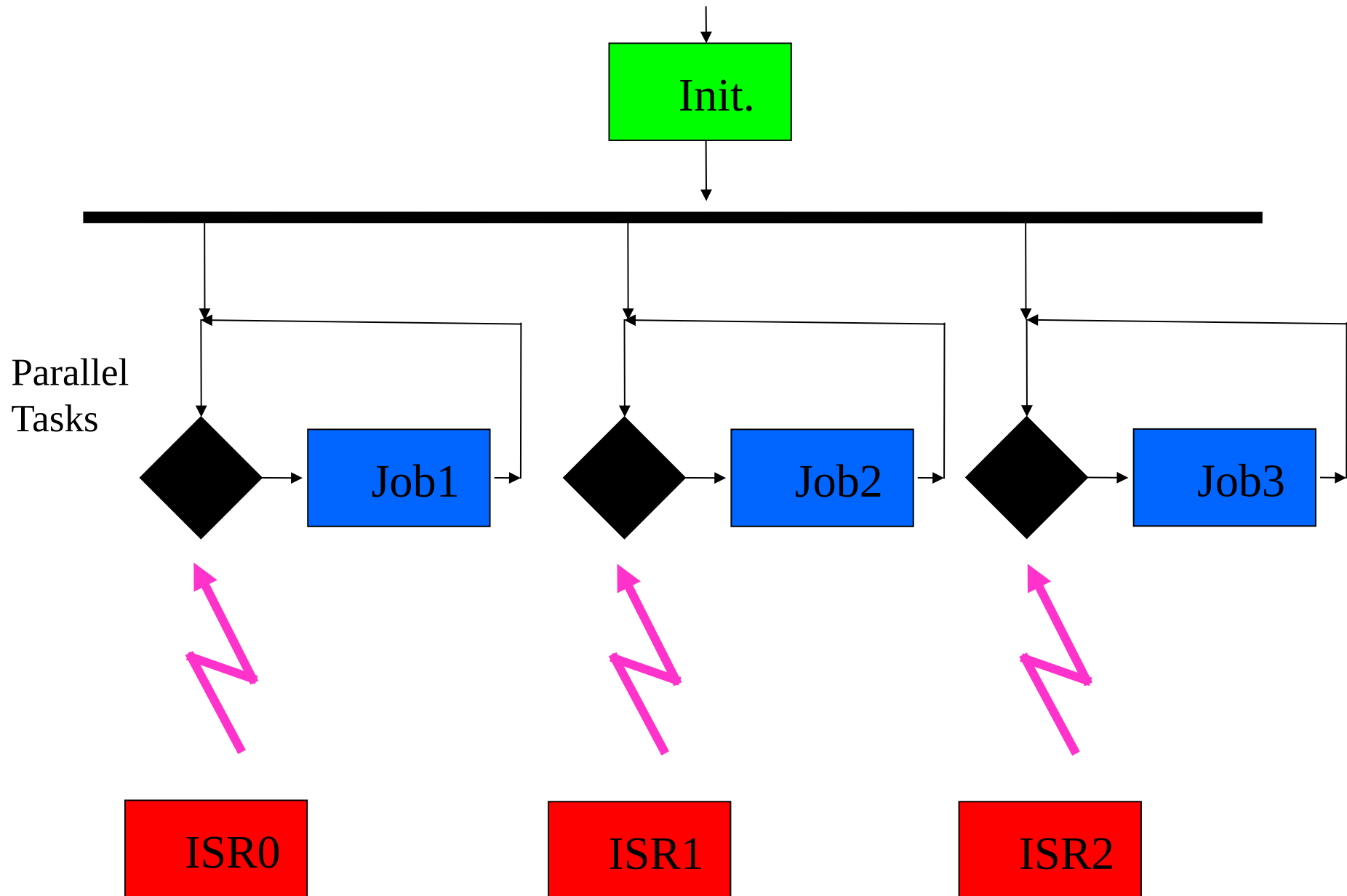


Multi-Threading

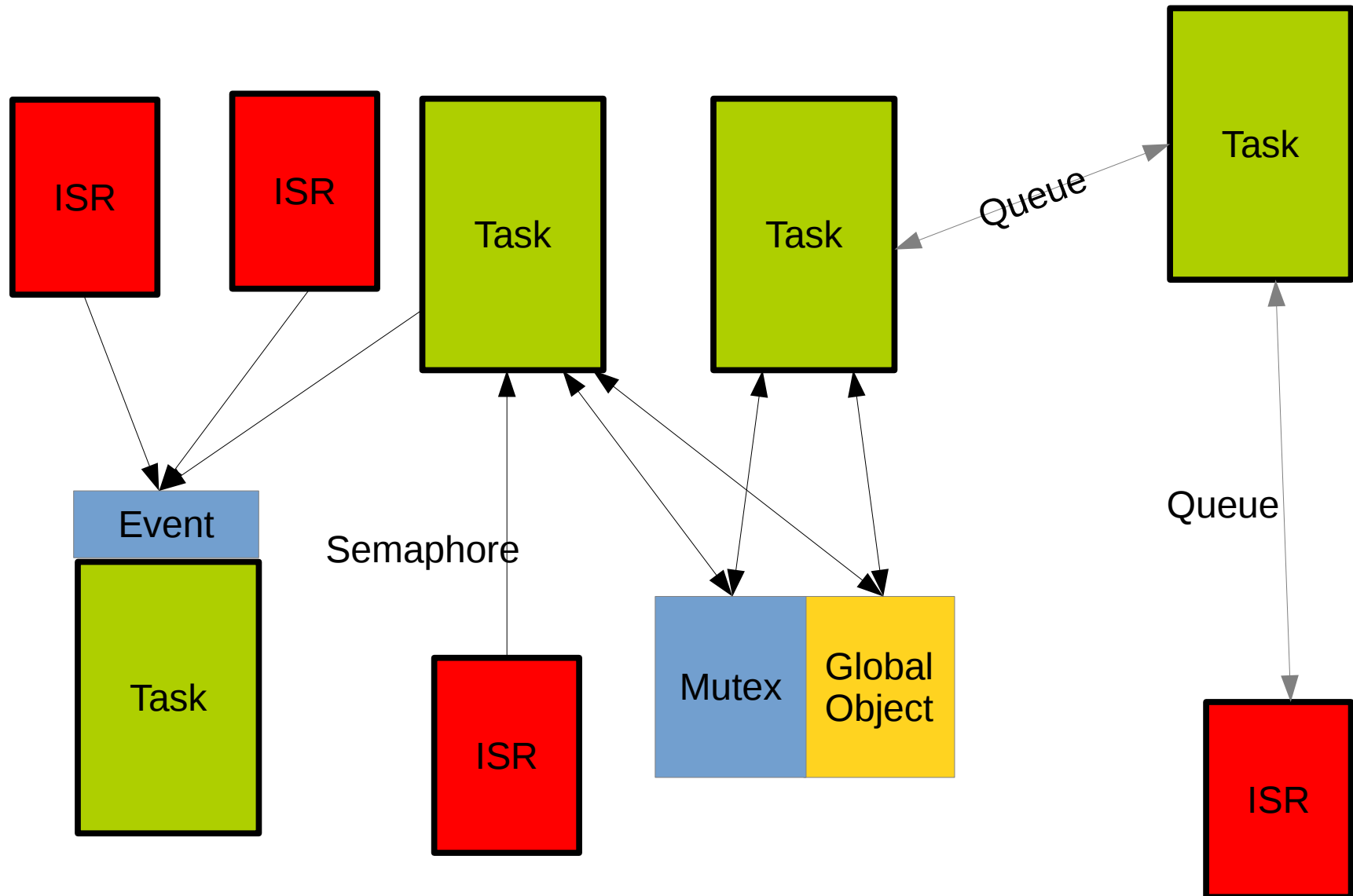


In some environments threads are called “tasks”.

Multitasking Interrupts



RTOS - System



Main() starts the RTOS ... nothing else !

```
int main(void)
{
    // Make sure to have correct vector table base address
    SCB->VTOR = (__IOM uint32_t)&g_pfnVectors;
    SCB->SHCSR |=
        (SCB_SHCSR_BUSFAULTENA_Msk
        | SCB_SHCSR_MEMFAULTENA_Msk
        | SCB_SHCSR_USGFAULTENA_Msk);

    SystemClock_Config();

    SCB_EnableICache();
    SCB_EnableDCache();

    #if configUSE_TRACE_FACILITY == 1
        vTraceEnable(TRC_START);
    #endif

    __HAL_RCC_PWR_CLK_ENABLE();
    __HAL_PWR_OVERDRIVE_ENABLE();
    __HAL_PWR_OVERDRIVESWITCHING_ENABLE();
    __HAL_RCC_SYSCFG_CLK_ENABLE();
    __HAL_FLASH_ART_ENABLE();
    __HAL_FLASH_PREFETCH_BUFFER_ENABLE();

    asm("b vTaskStartScheduler"); // start FreeRTOS, returns NEVER
}
```

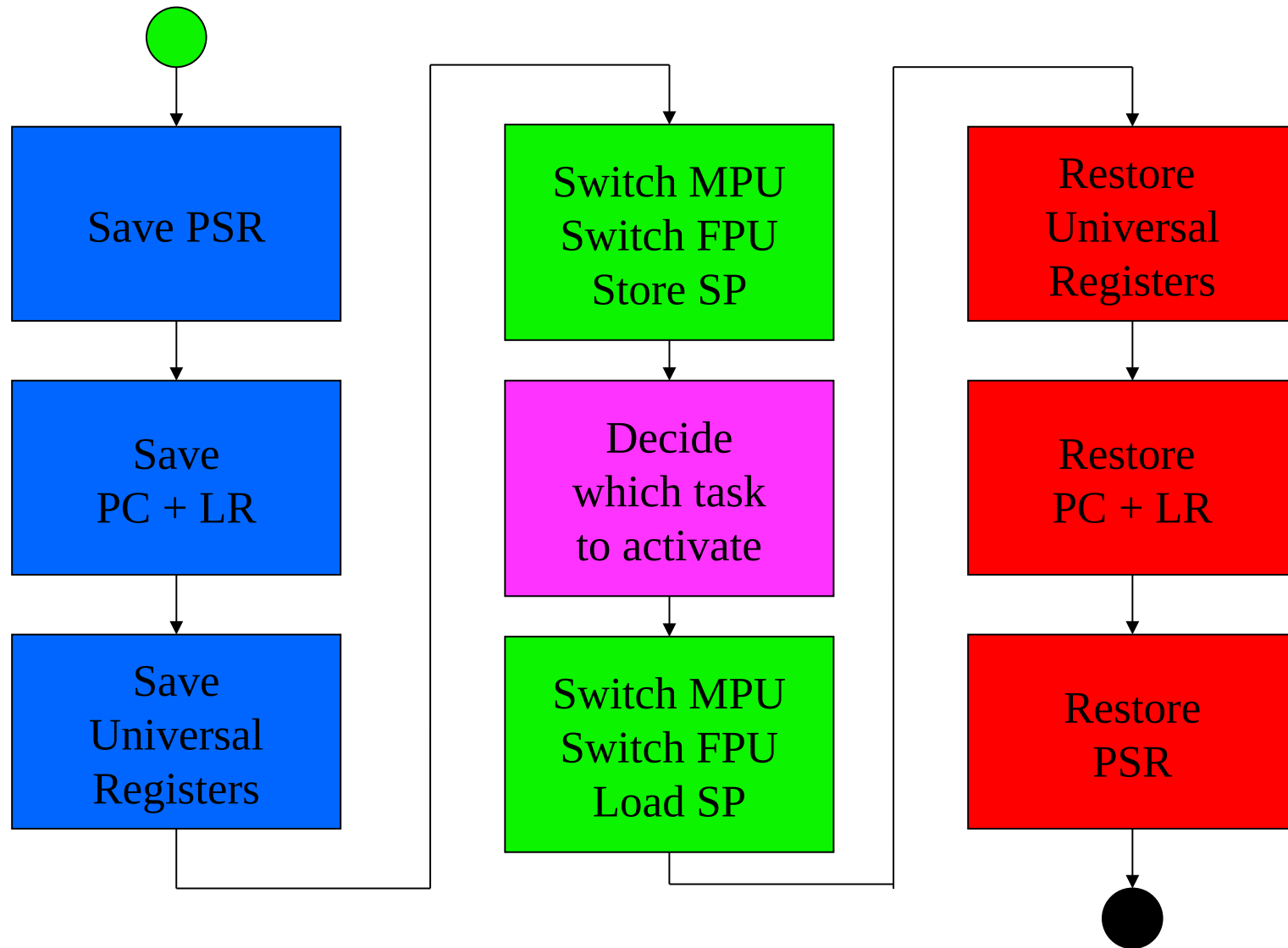


RTOS on Cortex M

RTOS Operation maps nicely to Cortex M operation modes:

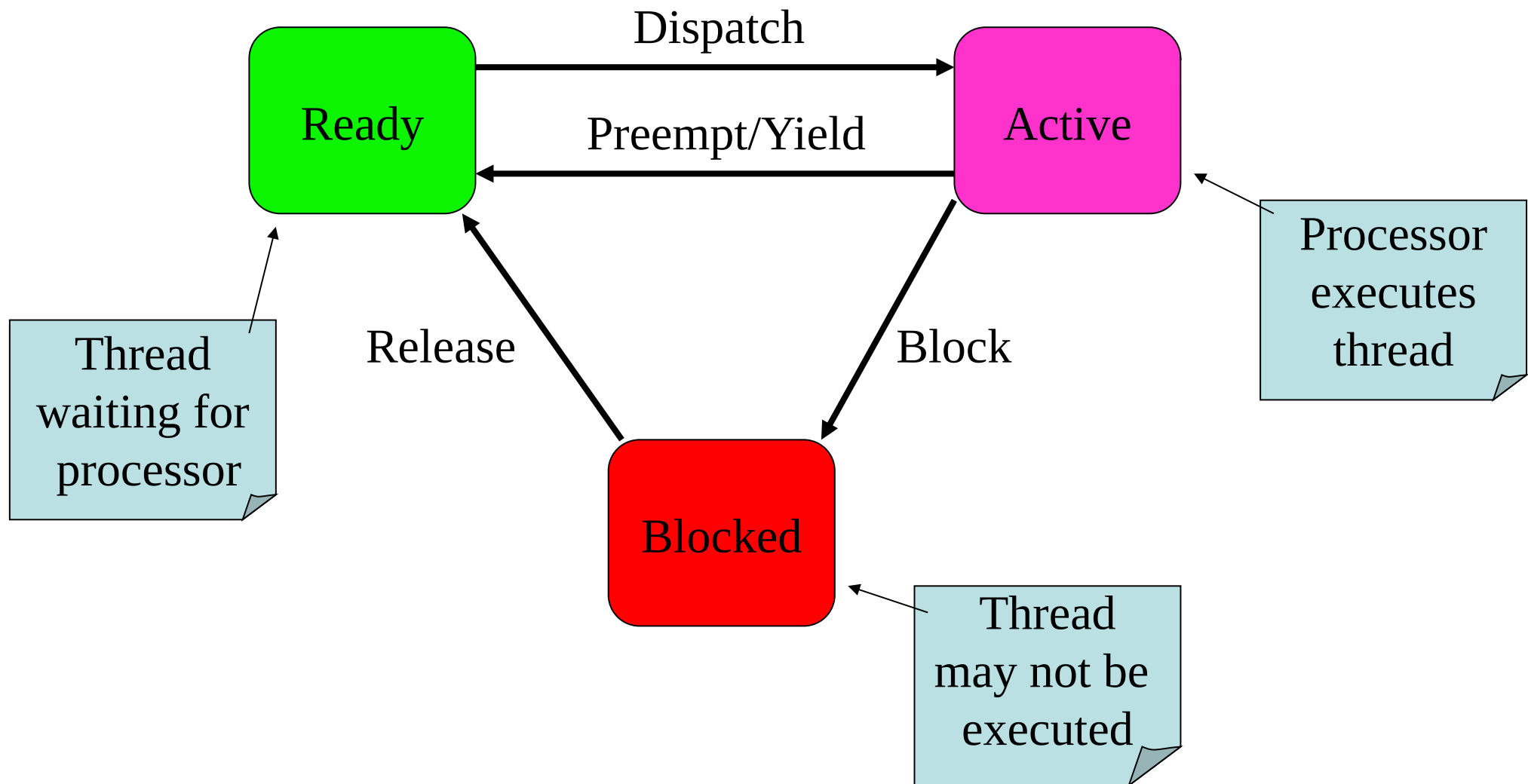
- Threads run in **user thread mode**
- System calls within threads may run in privileged thread mode if necessary
- Exceptions run in **privileged handler mode**
- Thread-switching is done using the SVC software interrupt
- PENDSV is used to perform thread switches inside system calls
- PENDSV is running as a low-priority exception to be interruptible

Thread Switch / Context Switch

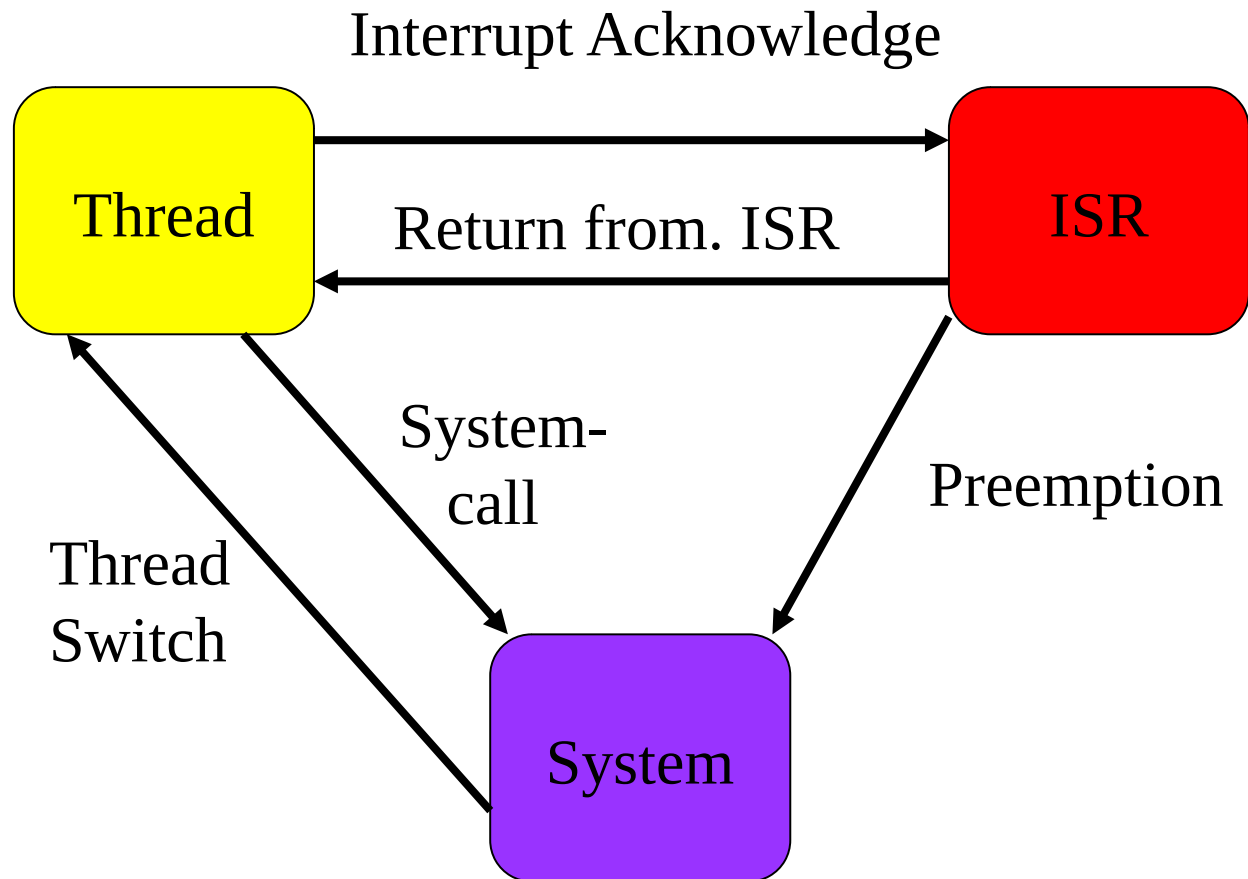


Cortex M Processors can do almost all of that automatically by using their SVC / return from interrupt mechanisms.

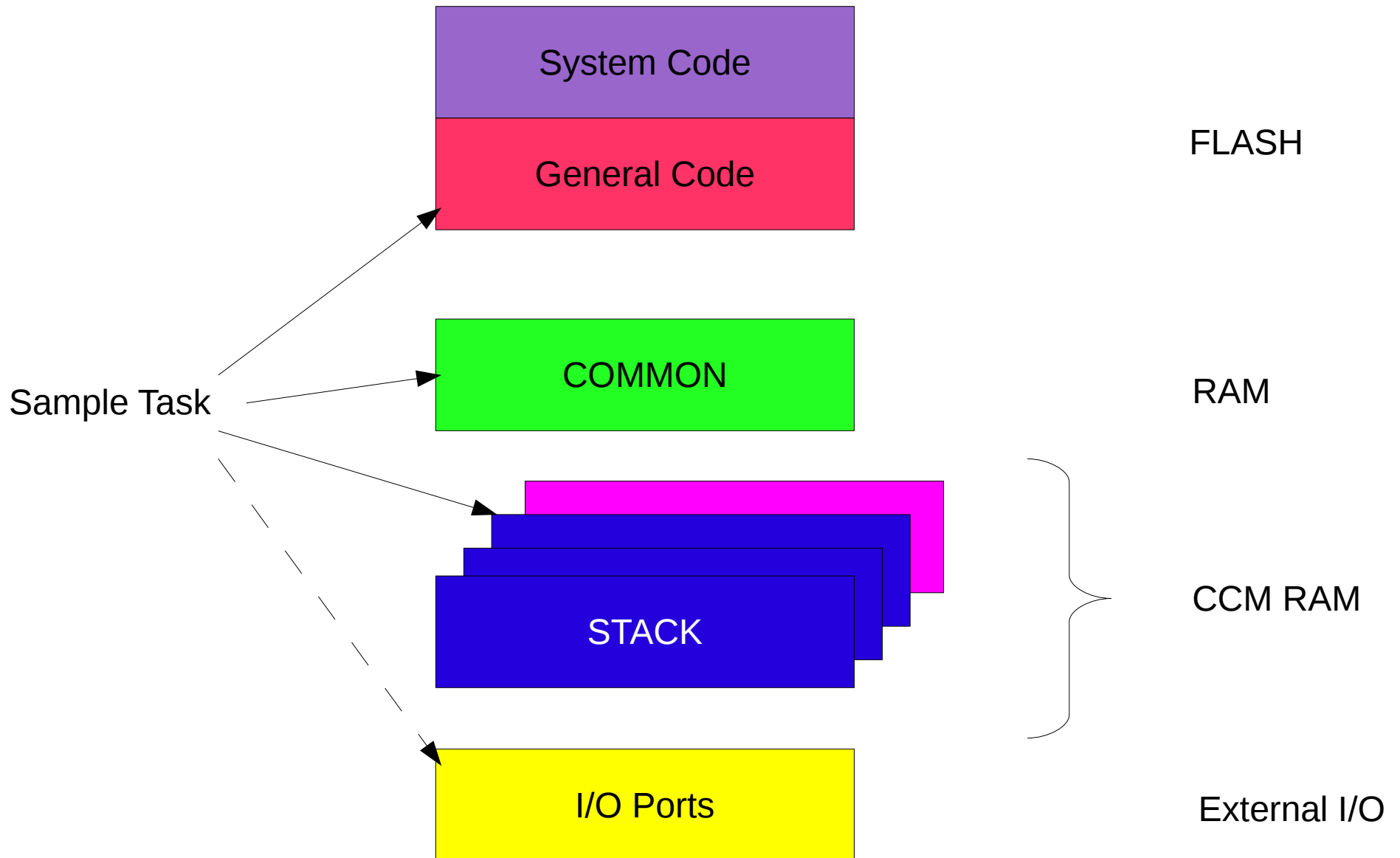
Thread State



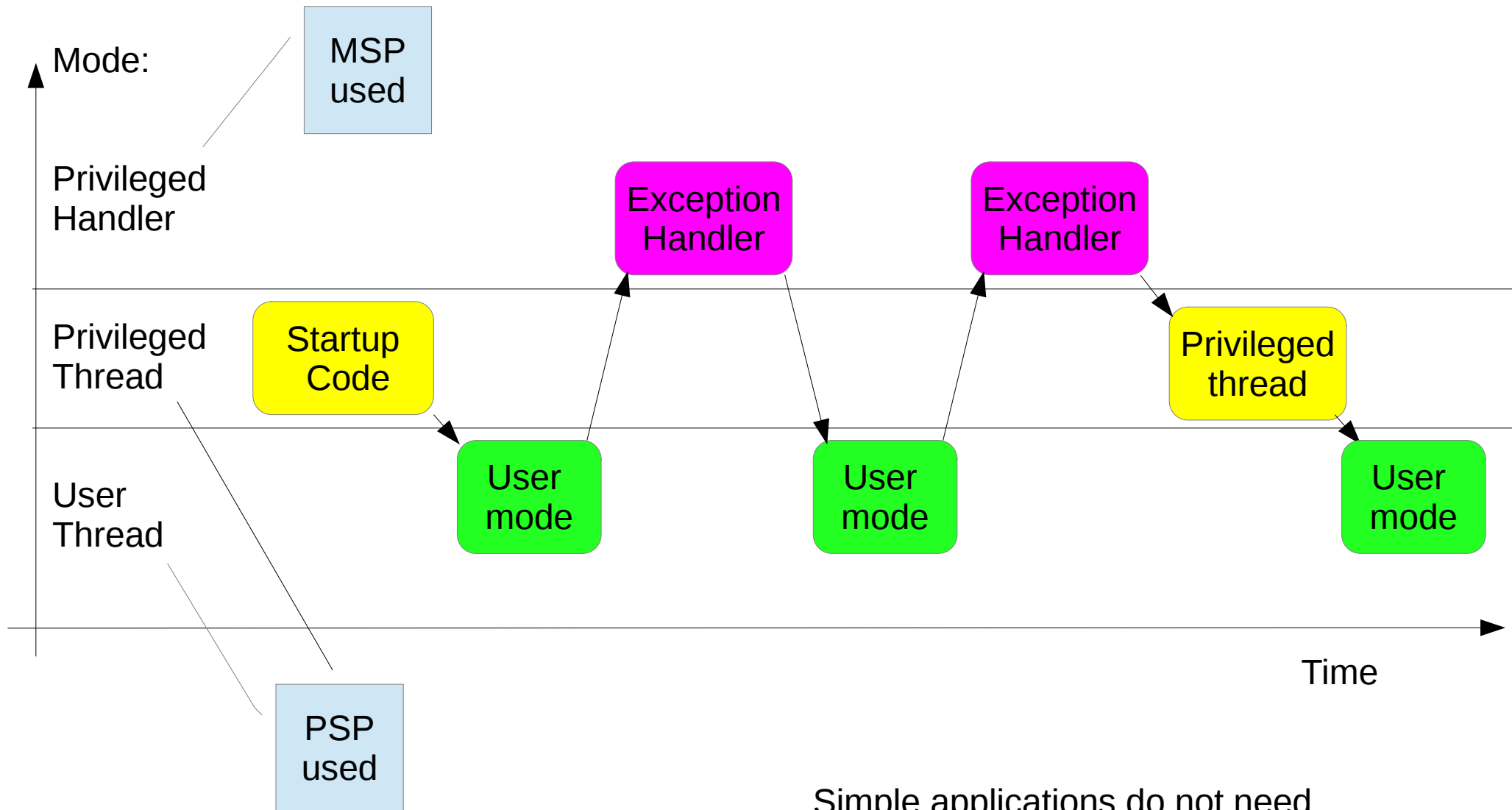
System State



Memory Layout



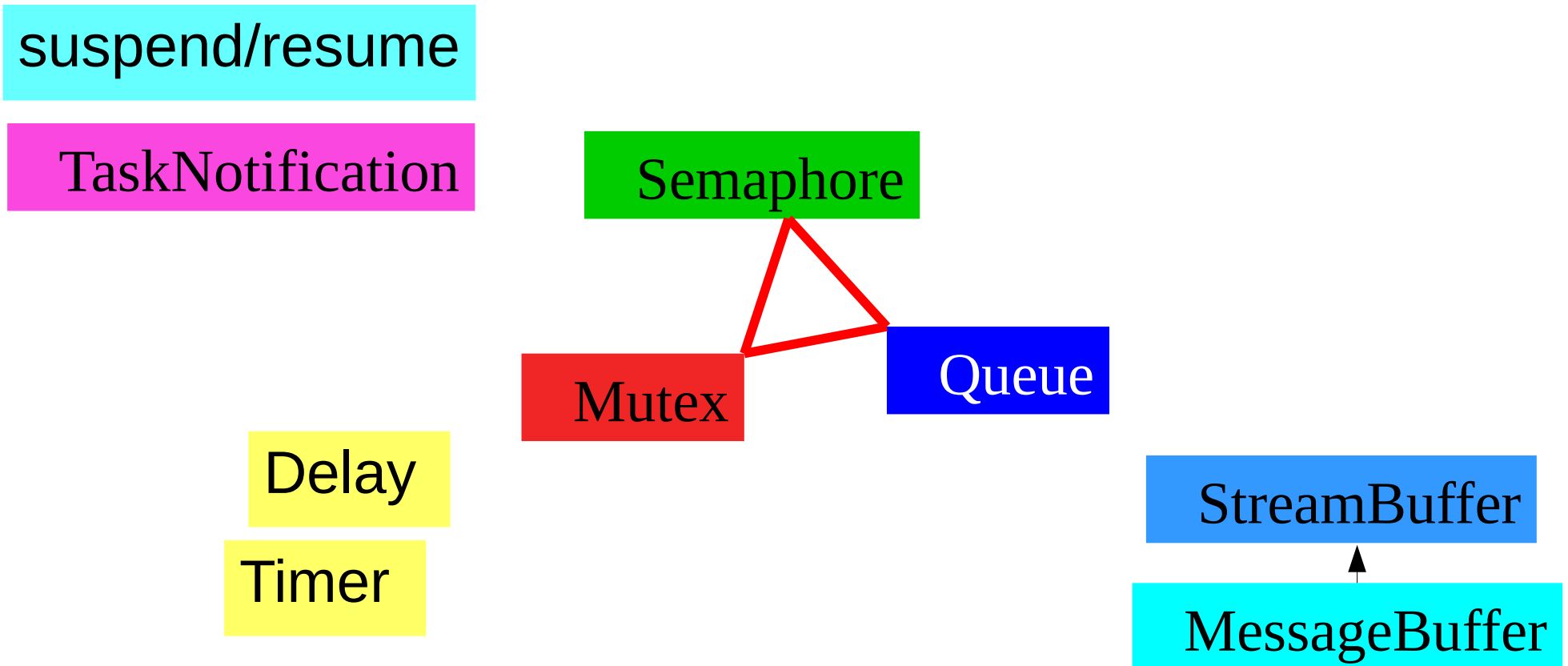
CM3 Operation Mode



Simple applications do not need to use the user thread (unprivileged) mode

Inter – Task Communication and Synchronisation

IPC = Inter – Process – Communication



Example: FreeRTOS

- Open-source real-time kernel (RTOS)
- Proven stability on many platforms
- Choice of preemptive or cooperative operation
- Ports for almost all micro-controllers available
- Small footprint within RAM and FLASH
- Good real-time behavior
- Run-time instrumentation for trace and debug available
- All basic mechanisms available
Mutex, Counting / Binary Semaphore, Queue, Timer etc.
- Interrupt / Task synchronization and communication mechanisms
- Sample installations for many evaluation boards ready for download

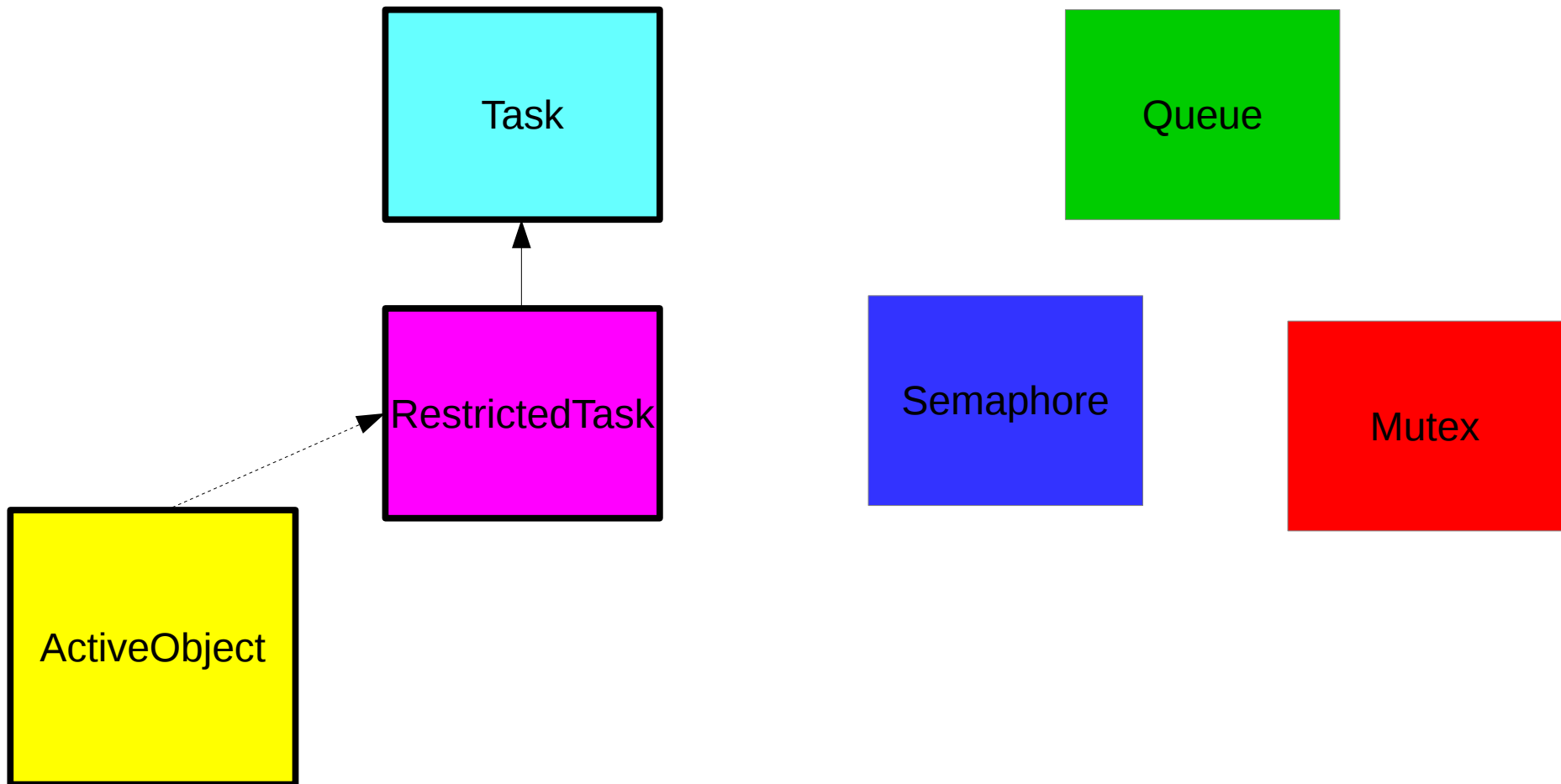


FreeRTOS API

API Reference

- Configuration
 - Task Creation
 - Task Control
 - Kernel Control
 - Task Notifications
 - Task Utilities
 - FreeRTOS-MPU Specific
 - Queue Management
 - Queue Sets
 - Semaphores
 - Software Timers
 - Event Groups
 - Stream Buffers
 - Message Buffers
 - Co-routine specific
-
- Task
- MPU - specific
- Queues
- Semaphores (binary, counting)
- Mutexes (normal, recursive)

FreeRTOS Wrapper Object Oriented Runtime Environment



Class Task

```
///! Task class
class Task
{
public:
    ///! Task constructor
    Task( TaskFunction_t runnable_code,
          char const * name          = "TSK",
          uint16_t stack_size        = configMINIMAL_STACK_SIZE,
          void * parameters          = 0,
          unsigned priority          = STANDARD_TASK_PRIORITY)
        : task_handle(0)
    {
        xTaskCreate(runnable_code, name, stack_size, parameters,
                    priority | portPRIVILEGE_BIT, &task_handle);
        ASSERT(task_handle != 0);
    }
    ///! suspend this task
    inline void suspend(void) const
    {
        vTaskSuspend(task_handle);
    }
    ///! Resume task execution, callable ONLY from task context !
    inline void resume(void) const
    {
        vTaskResume(task_handle);
    }
    ///! Resume task execution, callable ONLY from ISR !
    inline void resume_from_ISR(void) const
    {
        xTaskResumeFromISR(task_handle);
    }
private:
    TaskHandle_t task_handle; ///!< FreeRTOS's internal task handle
};
```

```

#include "system_configuration.h"
#include "FreeRTOS_wrapper.h"
#include "stm32746g_discovery.h"
extern "C" int trace_printf(...);

void task1_runnable( void *)
{
    BSP_LED_Init( LED1);
    while( true)
    {
        suspend();
        BSP_LED_Toggle( LED1);
    }
}

void task2_runnable( void *)
{
    int i=0;
    while( true)
    {
        suspend();
        trace_printf("hello nr. %d !\n", i++ * 500);
    }
}

Task task1( task1_runnable);
Task task2( task2_runnable);

void control_task_runnable( void *)
{
    synchronous_timer t(500);
    while( true)
    {
        task1.resume();
        t.sync();
        task2.resume();
        t.sync();
    }
}

Task task3( control_task_runnable);

```

Class “synchronous_timer”

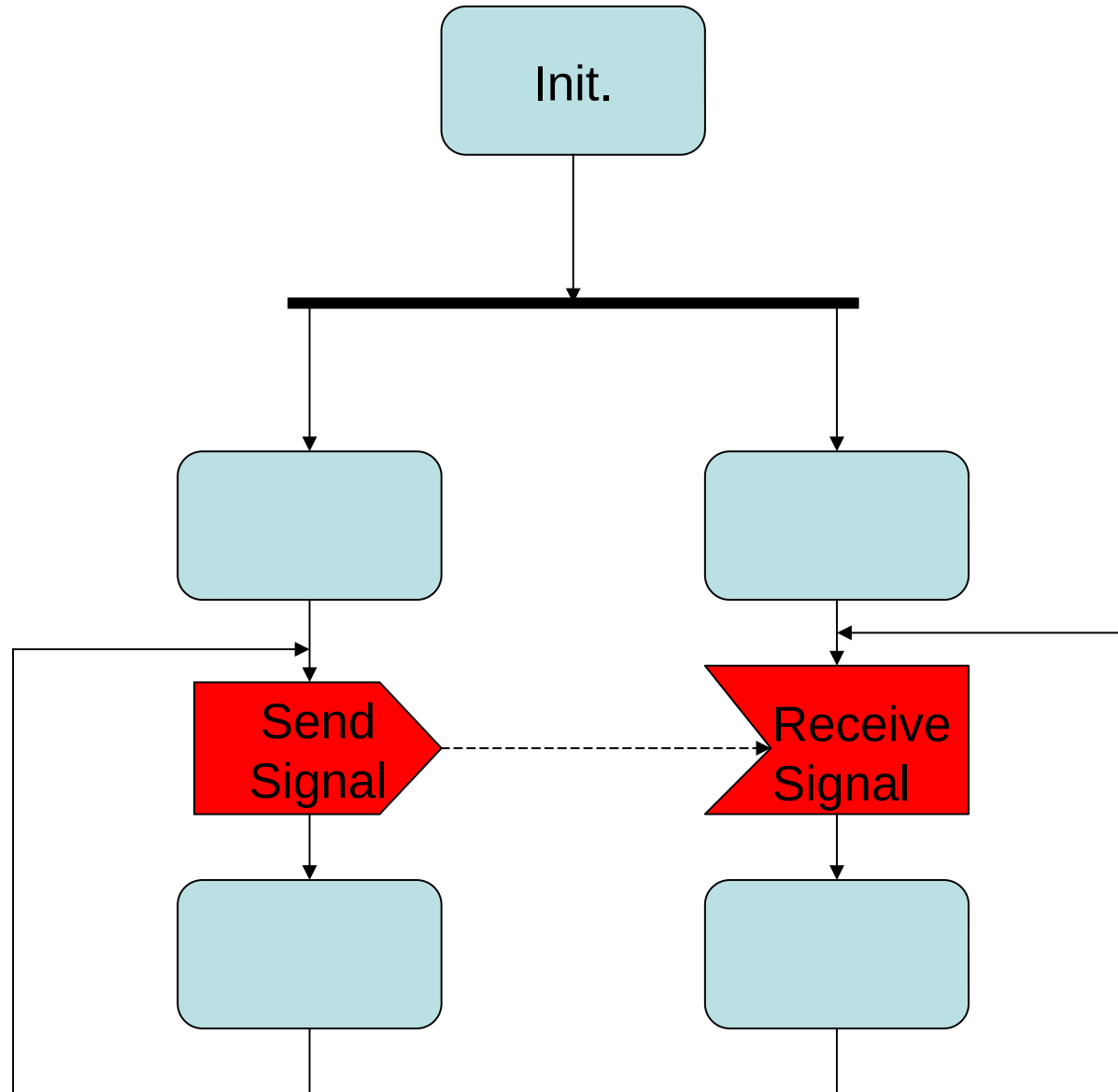
```
///! \brief wrapper around vTaskDelayUntil(...)
///!
///! Objects of this class allow precise periodic and long-term clock-synchronous activities
class synchronous_timer
{
public:
    synchronous_timer (TickType_t period = 0)
    :
        TimeIncrement (period),
        PreviousWakeTime ( xTaskGetTickCount ())
    {
    }
    ///! synchronize to periodic timer
    inline void
    sync (void)
    {
        ASSERT (TimeIncrement != 0);
        vTaskDelayUntil (&PreviousWakeTime, TimeIncrement);
    }
    ///! synchronous delay using DelayUntil(...)
    inline void
    delay (TickType_t duration)
    {
        vTaskDelayUntil (&PreviousWakeTime, duration);
    }
private:
    TickType_t TimeIncrement;
    TickType_t PreviousWakeTime;
};
```

Using synchronous_timer

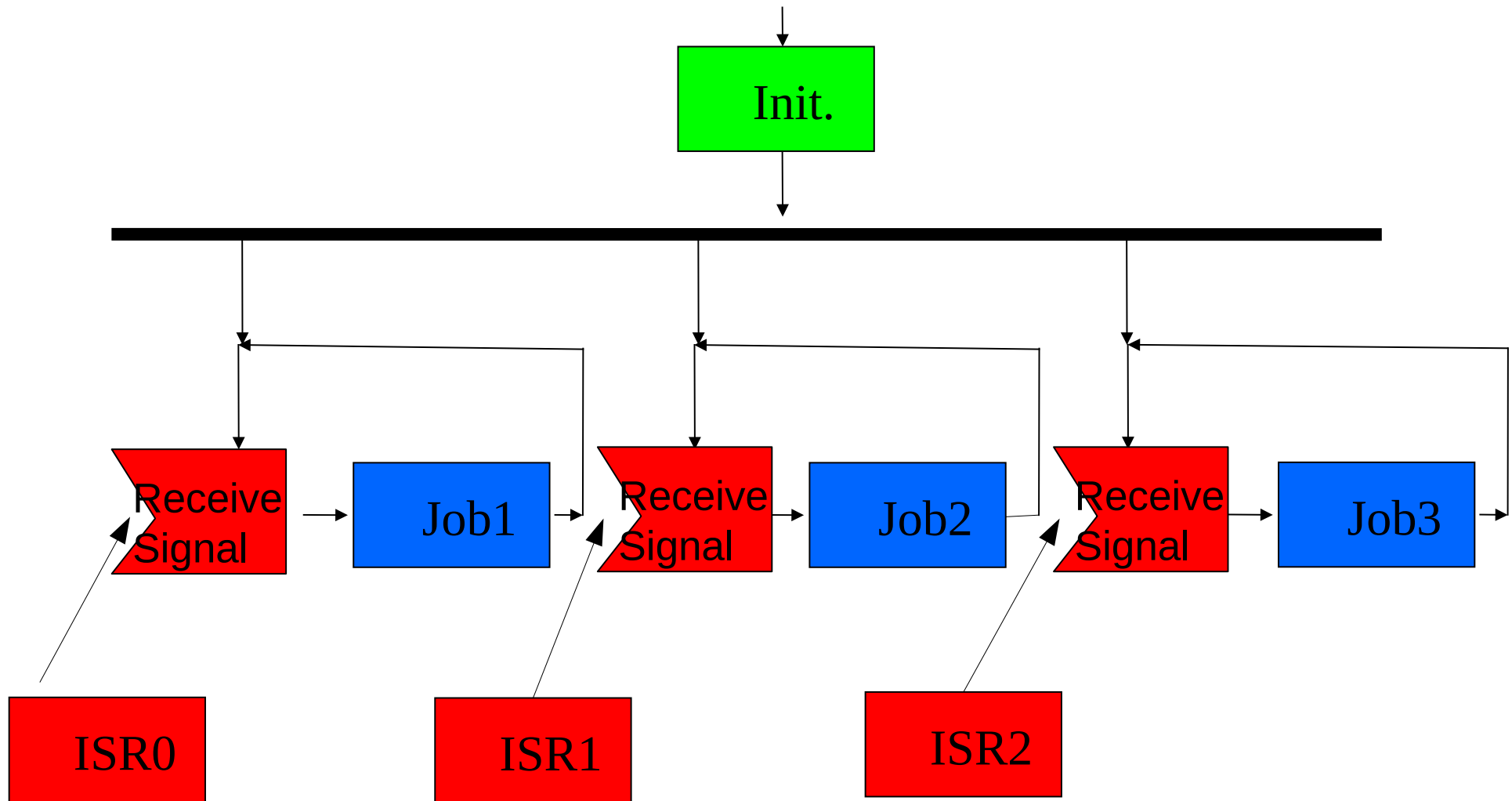
```
COMMON Semaphore task_3_synchronizer;

void
LED2_code (void *)
{
    BSP_LED_Init (LED2);
    synchronous_timer timer (333);
    while (true)
    {
        BSP_LED_Toggle (LED2);
        timer.sync ();
        BSP_LED_Toggle (LED2);
        task_3_synchronizer.signal ();
        timer.sync ();
    }
}
```

Inter-Task Synchronization

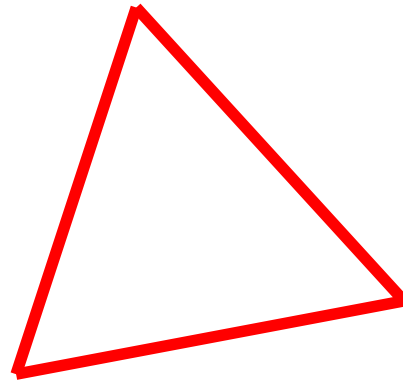


Multitasking Synchronized by Interrupts



“Magic Triangle”

Semaphore



Queue

Mutex

Queue

Useful for synchronization and data transfer.

Each queue has a constant configurable length and item size.

FreeRTOS calls include:

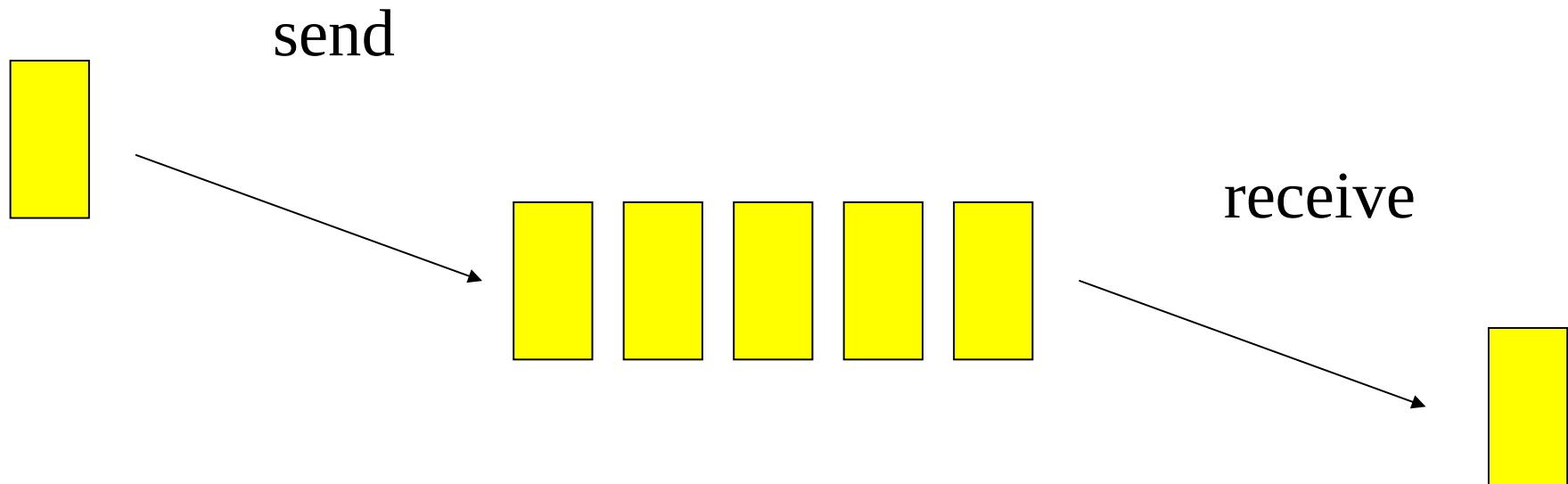
xQueueCreate create a queue (mandatory before use)

xQueueSend send data

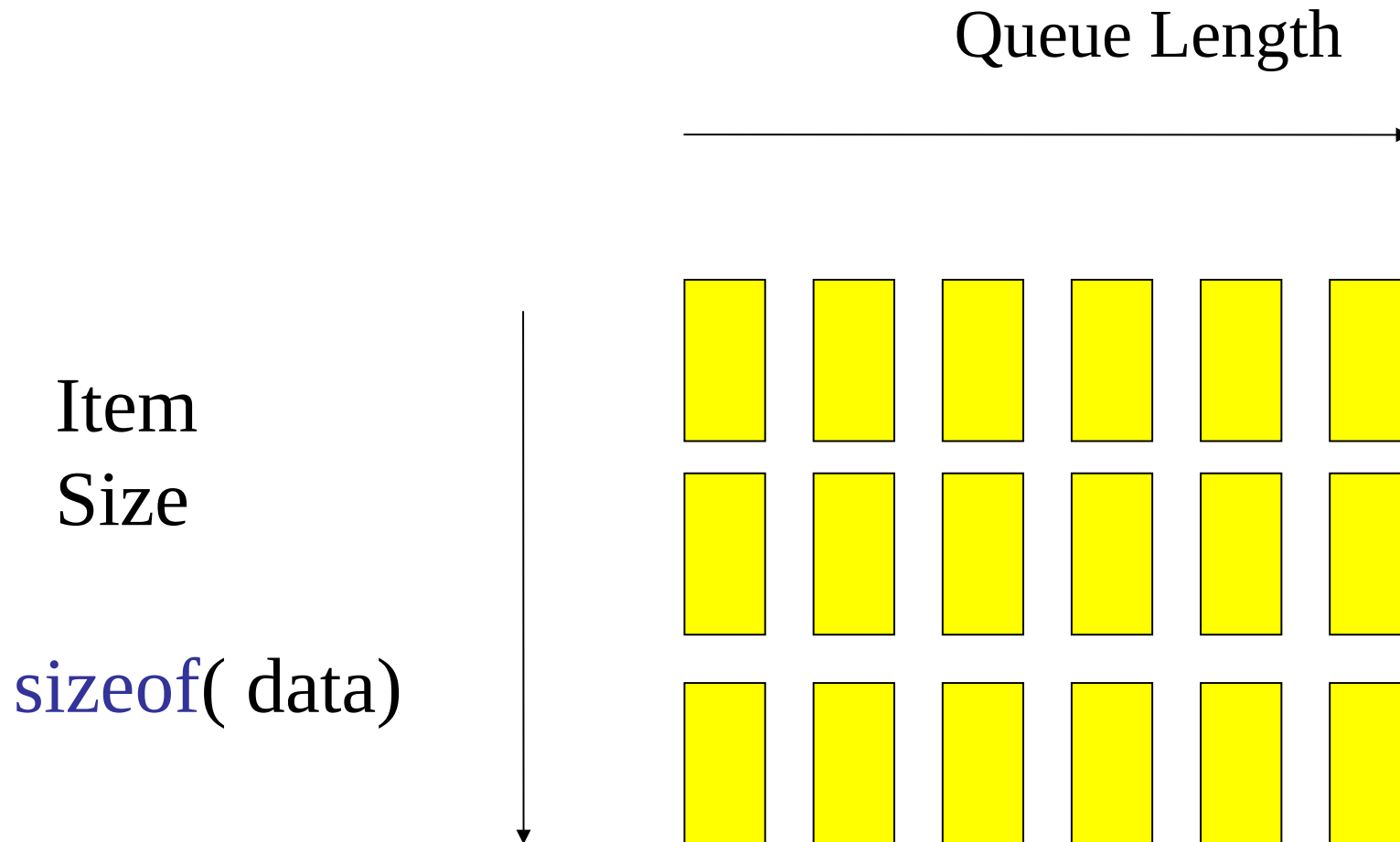
xQueueReceive receive data (or trigger)

A queue with item size zero
(i.e. lacking the capability for data transport)
is called a binary or counting semaphore.

Queue



Queue Dimensions



xQueueSend

[Queue Management]

queue.h

```
BaseType_t xQueueSend(  
    QueueHandle_t xQueue,  
    const void * pvItemToQueue,  
    TickType_t xTicksToWait  
);
```

xQueueReceive

[Queue Management]

queue.h

```
BaseType_t xQueueReceive(  
    QueueHandle_t xQueue,  
    void *pvBuffer,  
    TickType_t xTicksToWait  
);
```



```

template<typename items>
class Queue
{
public:
    ///! Queue constructor
    ///! \param length Number of items that can be stored
    Queue (unsigned length)
    {
        the_queue = xQueueCreate(length, sizeof(items));
        ASSERT(the_queue != 0);
    }
protected:
    // support for Semaphore wanting item size = 0
    ///! protected alternate Queue constructor, only for use by semaphores
    Queue (unsigned length, unsigned size)
    {
        the_queue = xQueueCreate(length, size);
        ASSERT(the_queue != 0);
    }
public:
    ///! Queue send method
    ///! \param item object to be sent
    ///! \param TicksToWait maximum time to wait (optional)
    inline bool send (items &item, unsigned TicksToWait = INFINITE_WAIT) const
    {
        return xQueueSend(the_queue, &item, TicksToWait) != pdFALSE;
    }

    ///! Queue send method for use within ISR's
    ///! \param item object to be sent
    inline void send_from_ISR (items item) const
    {
        BaseType_t task_woken=false;
        xQueueSendFromISR( the_queue, &item, &task_woken);
        portEND_SWITCHING_ISR( task_woken );
    }

    ///! Queue receive method
    ///! \param item reference to an object to be received, will be overwritten
    ///! \param TicksToWait maximum time to wait (optional ,default infinite wait)
    inline bool receive (items &item, unsigned TicksToWait = INFINITE_WAIT)
    {
        return xQueueReceive(the_queue, &item, TicksToWait) != pdFALSE;
    }
protected:
    QueueSetHandle_t the_queue; ///!< freeRTOS's Queue handle
};

```

Semaphore

Queue with item size zero

```
///! Counting and binary Semaphores
class Semaphore : private Queue<char> // type char is a dummy
{
public:
///! Semaphore constructor
///! \param length counting semaphore size (optional), length=1 or missing: create binary semaphore
Semaphore (unsigned length = 1) :
    Queue (length, 0)
{
}

///! signal method for use within tasks
///! \param TicksToWait maximum wait time (ticks)
inline bool signal (unsigned TicksToWait = NO_WAIT)
{
    return xQueueSend(the_queue, 0, TicksToWait) != pdFALSE;
}

///! signal method for use within ISR's
inline void signal_from_ISR (void) const
{
    send_from_ISR(0);
}

///! wait method
///! \param TicksToWait maximum wait time (ticks)
inline bool wait (unsigned TicksToWait = INFINITE_WAIT)
{
    return xQueueReceive(the_queue, 0, TicksToWait) != pdFALSE;
}
};
```

```

#include "FreeRTOS_wrapper.h"
#include "stm32746g_discovery.h"
#include "usart1driver.h"

COMMON Semaphore t1_synchronizer;
COMMON Semaphore t2_synchronizer;

⊖ void task1_runnable( void *)
{
    BSP_LED_Init( LED1);
    while( true)
    {
        t1_synchronizer.wait();
        BSP_LED_Toggle( LED1);
    }
}

⊖ void task2_runnable( void *)
{
    while( true)
    {
        t2_synchronizer.wait();
        delay(250);
        t1_synchronizer.signal();
    }
}

COMMON RestrictedTask task1( task1_runnable,"t1");
COMMON RestrictedTask task2( task2_runnable,"t2");

⊖ void control_task_runnable( void *)
{
    synchronous_timer t(500);
    while( true)
    {
        t1_synchronizer.signal();
        t.sync();
        t2_synchronizer.signal();
        t.sync();
    }
}

RestrictedTask task3( control_task_runnable, "control");

```

```
#include "FreeRTOS_wrapper.h"
#include "stm32746g_discovery.h"
#include "usart1driver.h"
```

```
COMMON usart1_driver usart1(115200);
```

```
void task1_runnable( void *)
{
    BSP_LED_Init( LED1);
    while( true)
    {
        Task::notify_take();
        BSP_LED_Toggle( LED1);
    }
}
```

```
void task2_runnable( void *)
{
    unsigned i=0;
    while( true)
    {
        Task::notify_take();
        usart1.puti(i++,10);
        usart1.newline();
    }
}
```

```
COMMON RestrictedTask task1( task1_runnable,"t1");
COMMON RestrictedTask task2( task2_runnable,"t2");
```

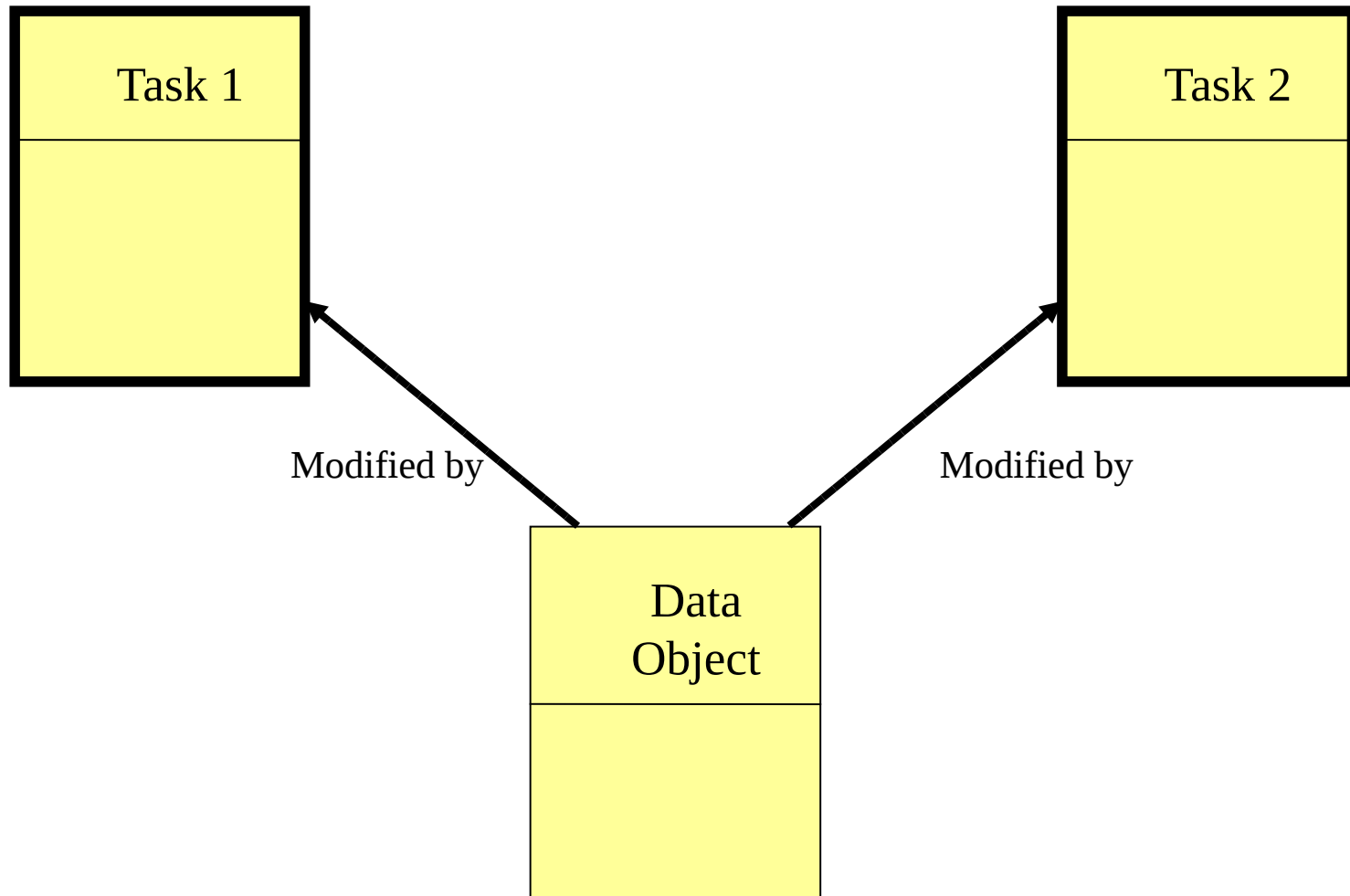
```
void control_task_runnable( void *)
{
    synchronous_timer t(500);
    while( true)
    {
        task1.notify_give();
        t.sync();
        task2.notify_give();
        t.sync();
    }
}
```

```
RestrictedTask task3( control_task_runnable, "control");
```

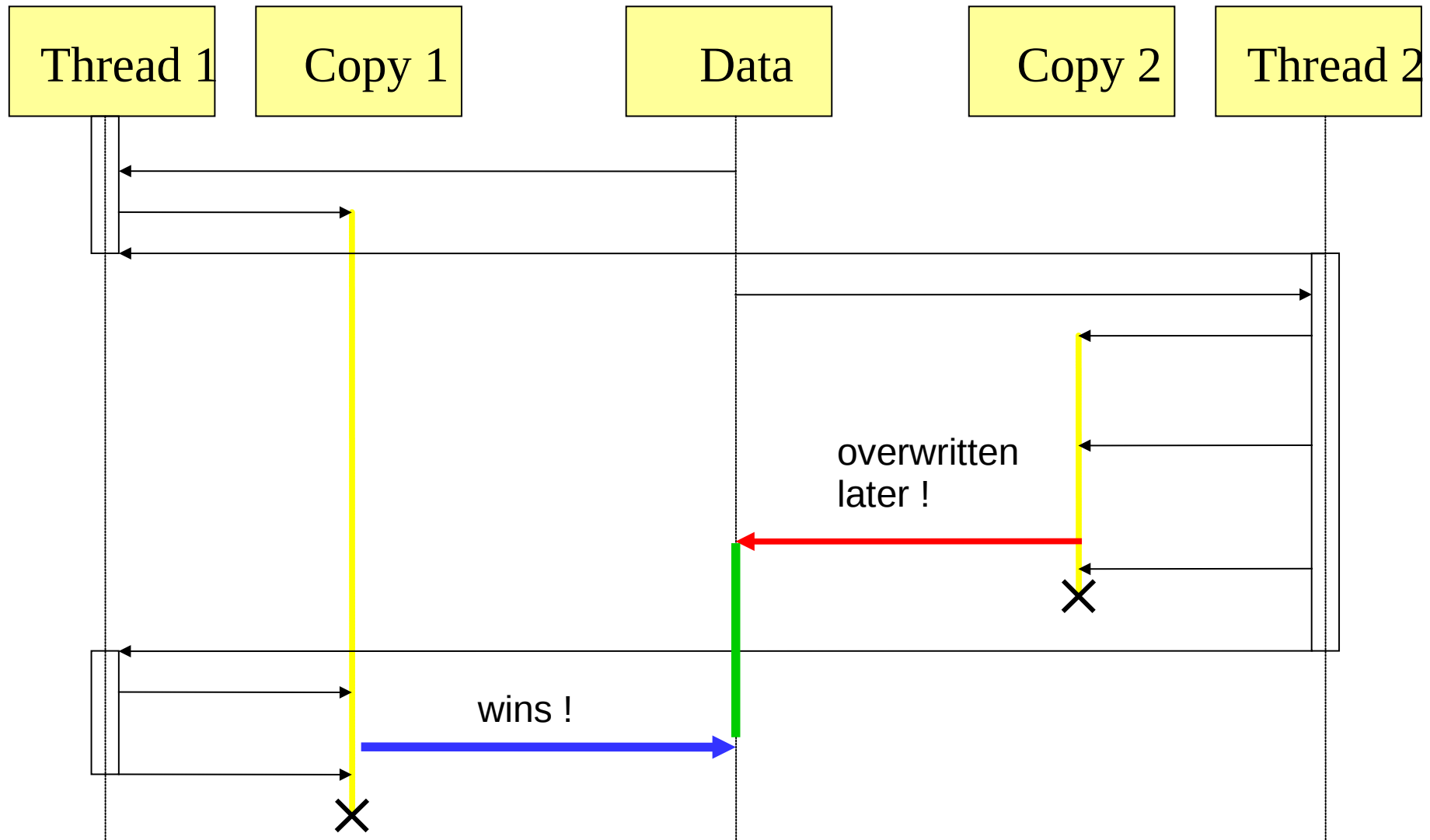
Task notification:
Lightweight task-associated
semaphore mechanism



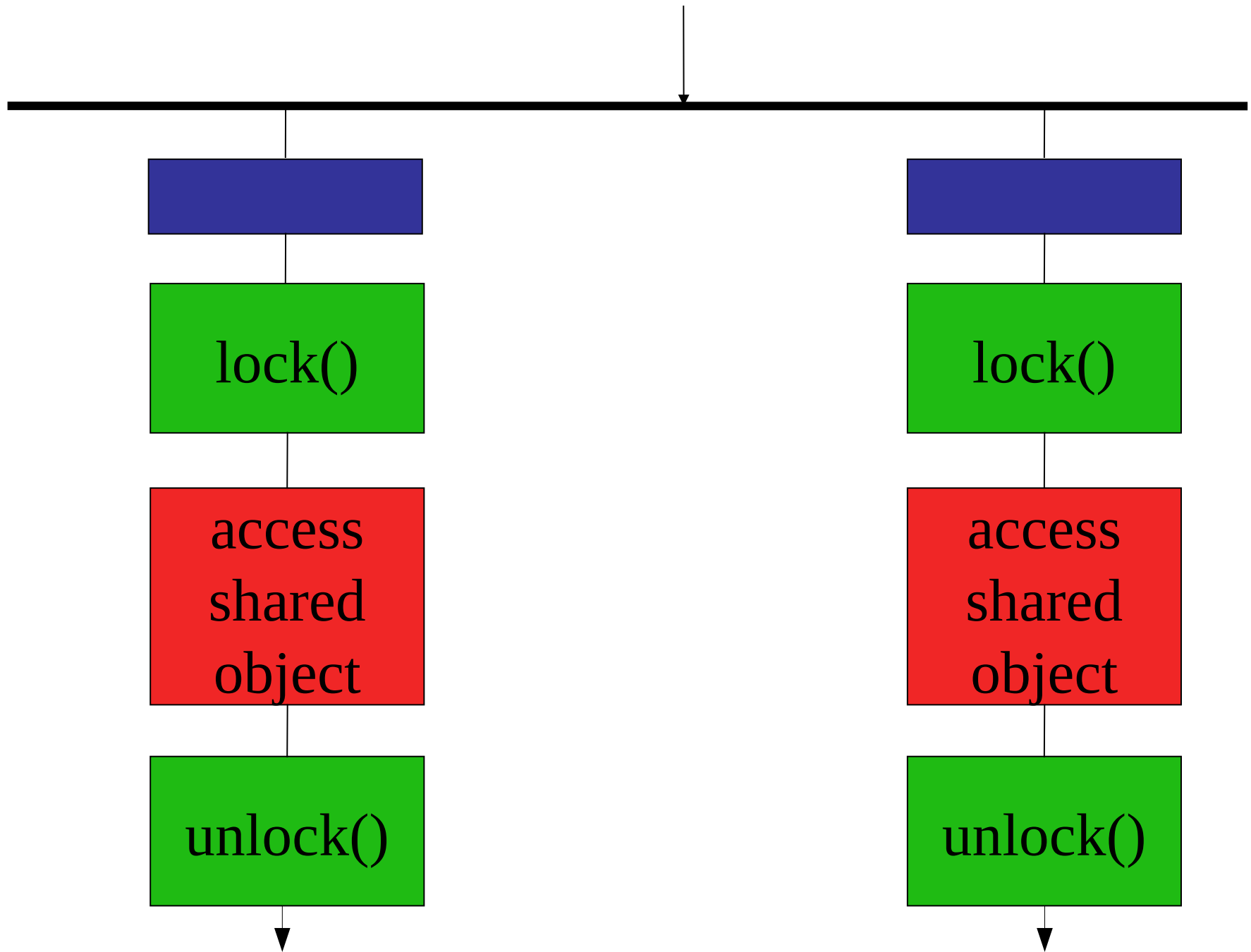
Shared Memory



Data Race



Critical Sections



Mutex

Useful for resource locking.

FreeRTOS supports mutexes with a priority inheritance mechanism.

FreeRTOS calls include:

xSemaphoreCreateMutex();

xSemaphoreTake();

xSemaphoreGive();

Recursive mutexes are also available.

```

#include "FreeRTOS_wrapper.h"
#include "stm32746g_discovery.h"
#include "usart1driver.h"

COMMON uint32_t somme;
COMMON uint32_t loops1;
COMMON uint32_t loops2;
COMMON usart1_driver usart1(115200);

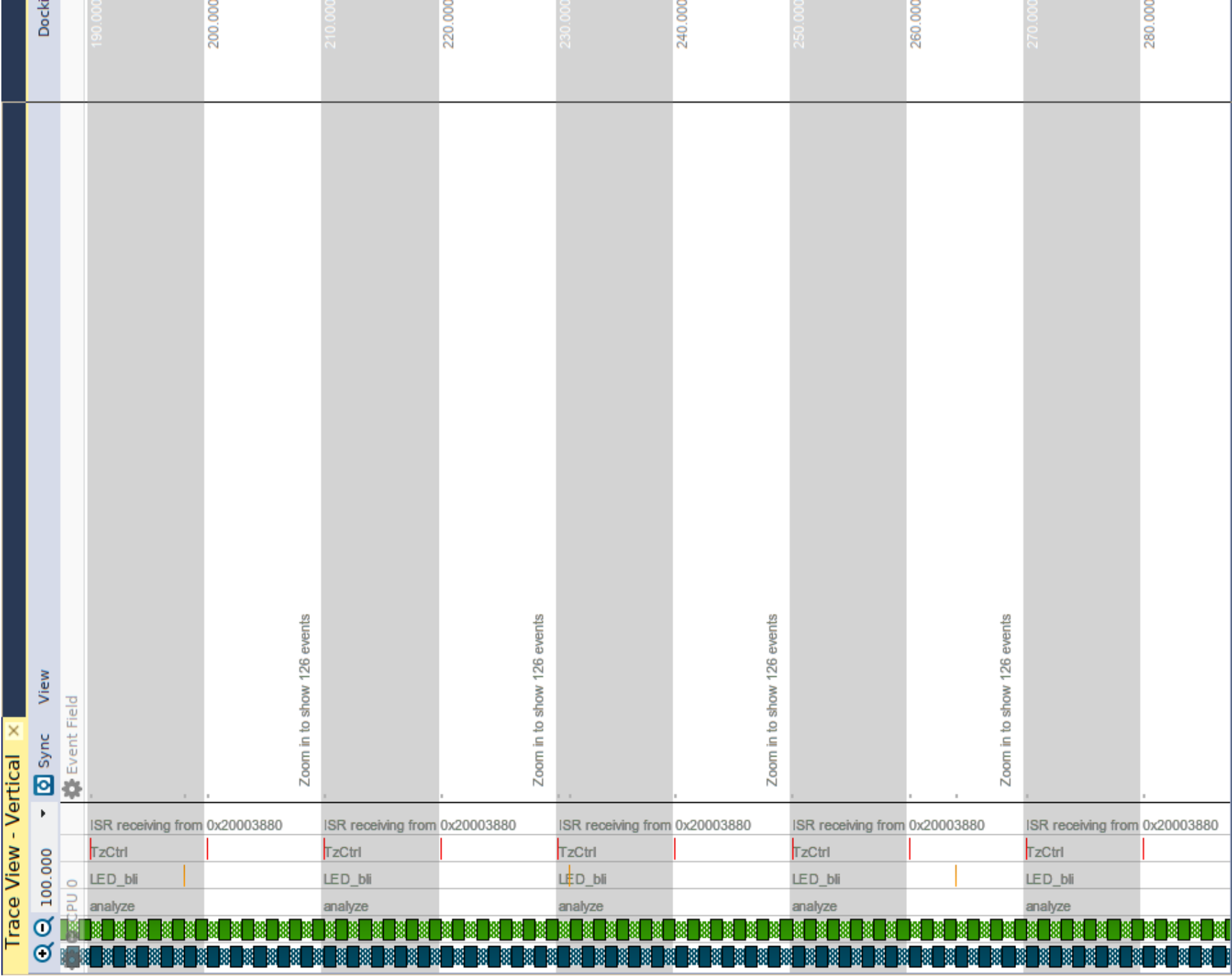
void task1_runnable( void *)
{
    while( true)
    {
        ++loops1;
        ++somme;
    }
}

void task2_runnable( void *)
{
    while( true)
    {
        ++loops2;
        ++somme;
    }
}

void analyzer_task_runnable( void *)
{
    uint32_t sum_old=0;
    for( synchronous_timer t(1000); true; t.sync())
    {
        usart1.puti( loops1);
        usart1.blank();
        usart1.puti( loops2);
        usart1.blank();
        usart1.puti( loops1+loops2-somme);
        usart1.blank();
        usart1.puti( loops1+loops2-sum_old);
        usart1.newline();
        sum_old=loops1+loops2;
    }
}

COMMON RestrictedTask task1( task1_runnable,"t1");
COMMON RestrictedTask task2( task2_runnable,"t2");
COMMON RestrictedTask task3( analyzer_task_runnable,
    "analyze",128,0,STANDARD_TASK_PRIORITY+2);

```



```

//! Mutex class
class Mutex
{
public:
    //! Mutex constructor
    Mutex (void)
    {
        the_mutex = xSemaphoreCreateMutex();
        ASSERT(the_mutex != 0);
    }
    //! Lock method for Mutex
    //! \param TicksToWait maximum time to wait to gain access (optional)
    inline bool lock (unsigned TicksToWait = INFINITE_WAIT)
    {
        return xSemaphoreTake(the_mutex, TicksToWait) != pdFALSE;
    }
    //! Release/Unlock method for Mutex
    inline void release (void)
    {
        xSemaphoreGive(the_mutex);
    }
    //! Release/Unlock method for Mutex
    inline void
    unlock (void)
    {
        xSemaphoreGive(the_mutex);
    }
private:
    SemaphoreHandle_t the_mutex; //!< FreeRTOS's SemaphoreHandle_t for the Mutex
};

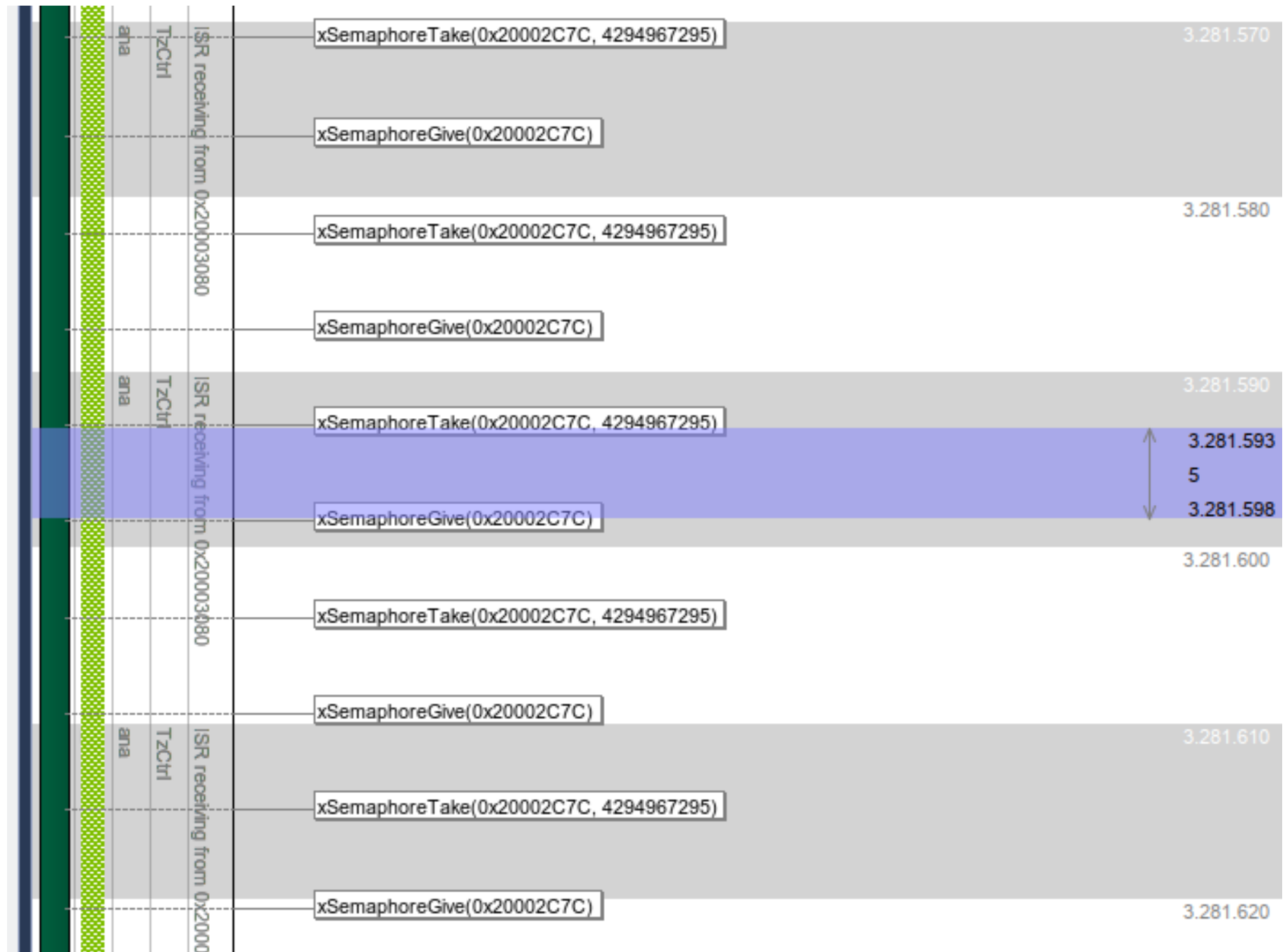
```

```
COMMON uint32_t sum;
COMMON uint32_t loops1;
COMMON uint32_t loops2;
COMMON usart1_driver usart1(115200);
COMMON Mutex sum_guard;

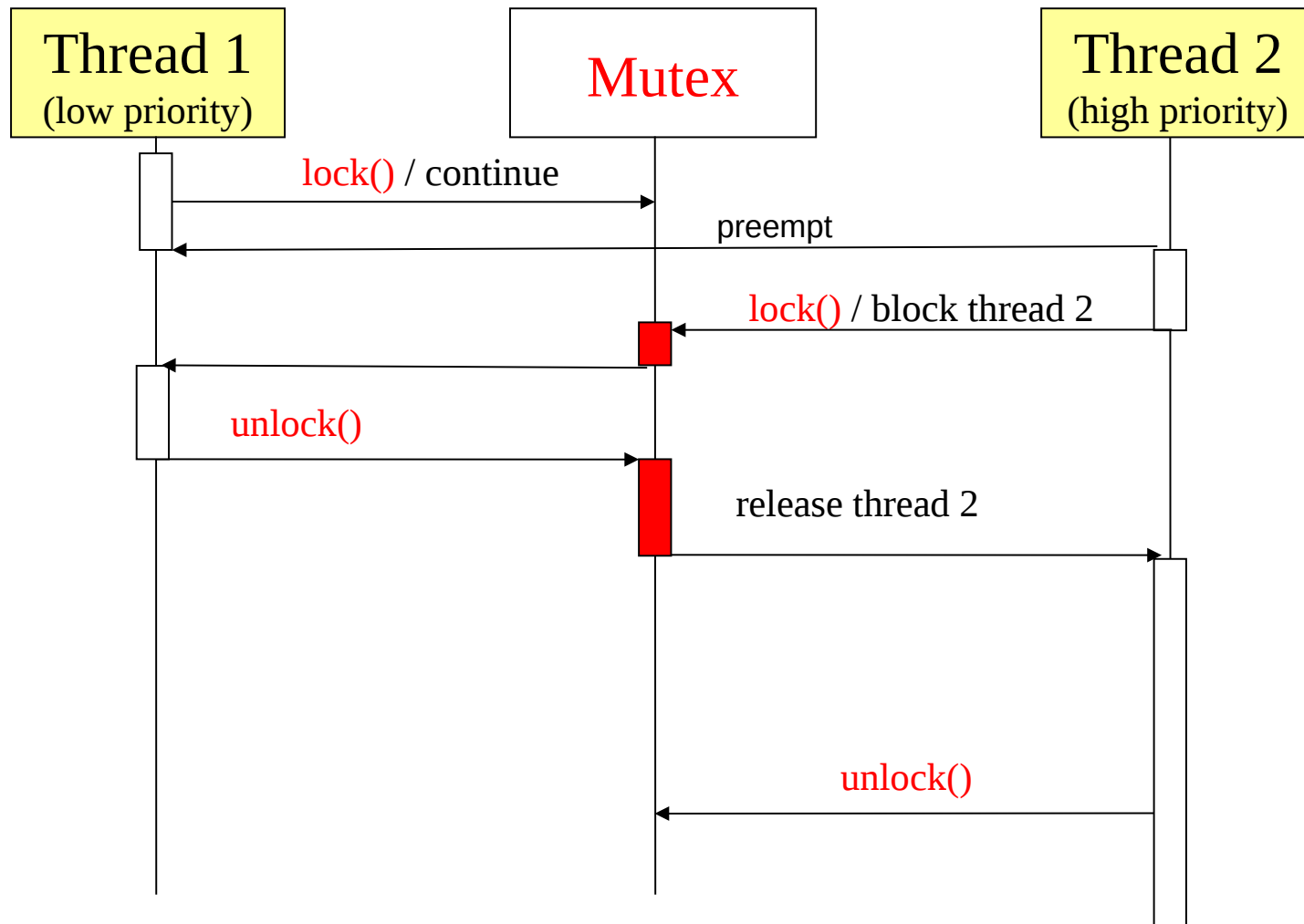
> void task1_runnable( void *)
{
    while( true)
    {
        ++loops1;
        sum_guard.lock();
        ++sum;
        sum_guard.release();
    }
}

> void task2_runnable( void *)
{
    while( true)
    {
        ++loops2;
        sum_guard.lock();
        ++sum;
        sum_guard.release();
    }
}
```

Mutex locking Overhead

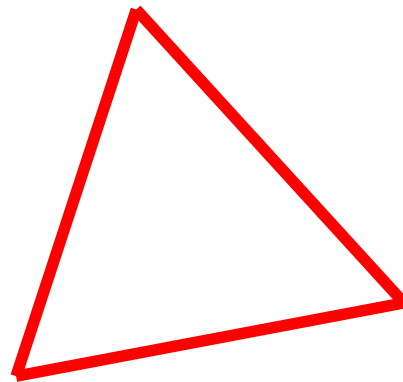


Mutual Exclusion



“Magic Triangle”

Semaphore



Queue

Mutex

Project for the STM32F7-Discovery Evaluation-Board

See also

Class **Task**



Class **RestrictedTask**

Class **active_object**

Class **Queue**

Class **Semaphore**

Class **Mutex**

Class **timer**

Class **synchronous_timer**

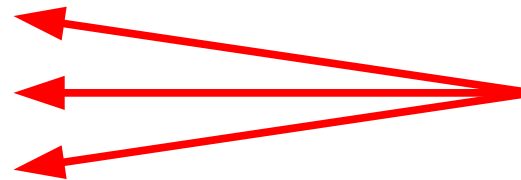
led_blinker

CAN_driver_t

usart3_driver

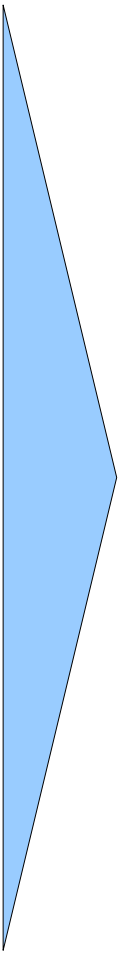
touch_queue

random_number_generator



Magic
Triangle

Lecture Overview

- 
- Introduction, Microcontroller basics
 - 32-bit Microcontroller technology
 - ARM Cortex M Architecture, (CM0 CM3 CM4 CM7)
 - Cortex M Microcontroller Instruction Set
 - Exception Handling, Interrupts
 - Real-time Operating Systems (RTOS)
 - Memory Management, protected memory, MPU
 - Implementing Calculations, Data Formats, FPU
 - I/O, Device drivers
 - Application Design
 - Case Studies, Advanced Design Patterns

<https://skript.eit.h-da.de/ams/>

